

✅ Here is the improved and enhanced Speaker Script for Session 2.

I have added in-depth, practical examples for better student understanding while keeping the language engaging and natural.

Slide 1: Title Slide

Script:

"Good morning everyone! Welcome back to **Session 2: Applied Prompt Patterns, Advanced Reasoning & Production Practices**.

In Session 1, we covered the foundations — moving from Chat UI to API, message roles, COSTAR framework, and structured outputs.

Today, we level up. We'll learn real-world prompt patterns, advanced reasoning techniques that make AI think smarter, and how to manage prompts in actual production systems. Let's get started!"

Slide 2: Session Agenda

Script:

"Here's what we have planned for this session.

We'll begin with Applied Prompt Patterns — the five most common tasks you'll use when building agents.

Then we'll dive into Advanced Reasoning Techniques like Chain of Thought and Self-Refine. Finally, we'll cover Prompt Management and Production realities.

We'll finish with a hands-on live exercise where we build a mini agent. Ready? Let's go!"

Slide 3: Quick Recap of Session 1

Script:

"Quick recap from Session 1: We learned that Chat UIs hide a lot of things, so serious work happens via API. We studied System, User, and Assistant roles, the powerful COSTAR framework, positive framing, structured outputs, and the iterative refinement loop. Everything we learn today builds on top of these fundamentals."

Slide 4: Applied Prompt Patterns Overview

Script:

"Most of the day-to-day work with LLMs falls into five major patterns. Today we'll study: Extraction, Classification, Transformation, Generation, and Decomposition.

Mastering these will help you use prompts more strategically when building AI agents."

Slide 5: Pattern 1 - Extraction

Script:

"**Extraction** is one of the most useful patterns.

Example: You have a long customer email or legal contract. You want the model to extract all important entities like names, dates, amounts, products, and action items.

A good extraction prompt clearly lists what to extract and in what format (usually JSON).

This is heavily used in invoice processing, resume parsing, and meeting note summarization."

Slide 6: Pattern 2 - Classification

Script:

"**Classification** helps us categorize input intelligently.

Real example: A user writes, 'My order hasn't arrived yet.' The model classifies the intent as 'Delayed Delivery' and routes it to the logistics team automatically.

Other uses: Sentiment analysis (Positive / Negative / Neutral), ticket categorization (Technical, Billing, Feature Request), or spam detection. Classification is the brain behind smart routing in agents."

Slide 7: Pattern 3 - Transformation

Script:

"**Transformation** means converting content from one form to another.

Example: Take a long, casual meeting transcript and transform it into a professional executive summary with action items and decisions.

Other examples: Translate English to Spanish, convert bullet points into a formal email, or

change a technical explanation into simple language for beginners."

Slide 8: Pattern 4 - Generation

Script:

"**Generation** is about creating new content from scratch.

Examples:

- Writing a full product launch email
- Generating Python code for a specific function
- Creating SQL queries from natural language
- Writing blog posts or social media content

The quality of generation heavily depends on how well you define role, context, and output format using COSTAR."

Slide 9: Pattern 5 - Decomposition

Script:

"**Decomposition** is a game-changer for complex tasks.

Instead of asking 'Analyze this business strategy,' you break it down:

1. First, extract key points.
2. Then, identify strengths and weaknesses.
3. Finally, give recommendations.

This pattern greatly improves accuracy and is the foundation of reliable multi-step agents."

Slide 10: Advanced Reasoning Techniques

Script:

"Now let's explore Advanced Reasoning Techniques — methods that make the model think more like a human expert."

Slide 11: Chain of Thought (CoT)

Script:

"The simplest yet most powerful technique is **Chain of Thought**.

Just add: 'Think step by step' at the end of your prompt.

Example: Instead of asking 'What is the capital of France?', you say:

'Answer the following question. Think step by step and explain your reasoning.'

This dramatically improves performance on math, logic, and multi-step problems."

Slide 12: Self-Consistency

Script:

"**Self-Consistency** takes Chain of Thought further.

You generate 3 to 5 different reasoning paths (by setting temperature > 0), then take the majority answer.

Example: For a tricky math or reasoning question, running it 5 times and choosing the most consistent answer significantly reduces hallucinations."

Slide 13: Self-Refine

Script:

"**Self-Refine** is excellent for improving output quality.

The process has three steps:

1. Generate initial answer
2. Critique: 'Review your previous answer. What are the weaknesses?'
3. Refine: 'Now improve your answer based on the critique.'

This loop produces much higher quality final outputs."

Slide 14: Least-to-Most Prompting

Script:

"**Least-to-Most Prompting** is very effective for hard problems.

"You start by asking a simple question, then gradually increase the complexity of the question as you go."

you solve the easiest parts first, then use those answers to solve harder ones.

Example: To solve a complex word problem, first ask the model to identify all given variables, then solve sub-equations, and finally combine them."

Slide 15: Tree of Thoughts (ToT)

Script:

"**Tree of Thoughts** is a more advanced method.

The model explores multiple different reasoning paths (like branches of a tree), evaluates each path, and chooses the most promising one.

It's very powerful for planning, coding, or strategic problems, but also more expensive."

Slide 16: Prompt Management in Production

Script:

"Creating good prompts is only half the battle. Managing them in production is equally important."

Slide 17: Prompt Versioning

Script:

"**Prompt Versioning** means treating prompts like code.

You should version them (v1, v2, v3) so you can track which version performs better.

You can store them in code files or use dedicated prompt management tools."

Slide 18: A/B Testing Prompts

Script:

"**A/B Testing** is how top teams improve prompts continuously.

Example: You have two versions of a customer support prompt. You run both with real users for a week and measure which one gives higher customer satisfaction or fewer escalations."

Slide 19: Prompt Caching & Cost Optimization

Script:

"**Prompt Caching** is a recent powerful feature.

If your system prompt or product documentation doesn't change often, you can cache it.

This saves money and makes responses faster.

Other tips: Use smaller models for simple tasks and monitor token usage carefully."

Slide 20: Live Exercise

Script:

"Now comes the fun part — our **Live Exercise**!

We're going to build a Mini Research & Analysis Agent that uses Chain of Thought, Decomposition, and Self-Refine together.

This is where everything we learned today comes together. Please open your Python project and let's build it step by step."

Slide 21: Key Takeaways + Thank You

Script:

"Key Takeaways from Session 2:

1. Master the five Applied Prompt Patterns.
2. Use Chain of Thought and Self-Refine to boost reasoning quality.
3. Break complex tasks using Decomposition.
4. Treat prompting as a serious engineering practice — version, test, and optimize.

Thank you so much for your active participation across both sessions!

You now have a strong foundation to build real AI Agents.

I'm happy to take any questions. Feel free to reach out anytime!"

This version is now much richer with **practical examples** while staying within reasonable time limits.

Would you like me to:

- Add timing suggestions for each slide?
- Move to **Live Exercise** design next?
- Prepare the Jupyter Notebook / Python project for Session 2?

Just tell me what's next! 🚀